

Redesigning Web Application Architecture for Scalability and Maintainability: A Case Study

1. Abstract

This research article conducts an examination of web application architectural redesign solutions for scalability and maintainability challenges caused by growing user numbers. This research compares three architectural designs including model-view-controller (MVC), event sourcing, and microservices. MVC architecture enhances system modularity and testability while ES architecture offers auditability and resilience through its immutable event log and microservices architecture enables autonomous scalability and fault separation. The suggested method resolves monolithic architecture issues by merging these approaches to enhance scalability over time while improving maintainability and user satisfaction.

2. Introduction

As the number of users for web applications increases the architecture demands testing and revision. As an online opinion-sharing platform grew quickly its architecture showed weaknesses because it relied on one database and interconnected code that restricted scaling and performance capabilities.

The monolithic architecture resulted in scalability problems and slower web page performance along with development collaboration challenges. The job of test engineers becomes difficult when they try to detect problems and write unit tests which leads to drops in system stability and maintainability.

This research conducts a thorough examination of Event Sourcing, Model-View-Controller (MVC), and Microservices to tackle these architectural issues. The study introduces a single comprehensive solution for converting monolithic architectures into modular systems that can scale effectively while maintaining user satisfaction over time.

3. Architecture of Model-View-Controller (MVC)

3.1 Overview and Key Concepts

The separation of Model, View, and Controller components in the Model-View-Controller (MVC) architecture improves testability, scalability, and maintainability of web applications by enabling modular development.

- **Model:** The Model manages business logic and data processing through its database interactions. The system fetches and confirms data as well as updates it while maintaining a reusable structure and reducing workload through data logic abstraction.
- **View:** The View layer provides user interface data presentation while enabling separate design modifications without interfering with the business logic.
- **Controller:** The Controller functions as a mediator between Model and View by processing user inputs and executing business logic to enable smooth transitions.

3.2 Relevance to the Application Context

In modern applications, where scalability and user engagement are vital, MVC proves essential:

- **Separation of Concerns:** The MVC framework divides user interface responsibilities from business logic and user interaction handling to prevent problems associated with monolithic tightly coupled code.
- **Parallel Development:** The modular structure of MVC allows teams to simultaneously develop different components which leads to shorter development cycles and fewer merge conflicts.
- **Improved Testability:** MVC makes unit testing easier because it separates components so developers can make accurate test cases and fix errors more quickly.

3.3 Performance Analysis by Characteristics

The MVC design successfully addresses the following key performance factors:

- **Scalability:** Each component tier may extend independently, allowing the Model to manage more data while keeping the Controller and View unaffected.

Redesigning Web Application Architecture for Scalability and Maintainability: A Case Study

- The modular architecture of unit testing frameworks ensures comprehensive coverage for all components.
- **Data Consistency:** Storing all actions in the Model ensures consistent data management throughout the application.
- Clear duties promote teamwork, reduce disagreements, and speed up the incorporation of new developers.

3.4 Implementation in the New Version

Key performance factors are effectively addressed by MVC:

- Choose an ORM solution such as Hibernate or Sequelize to simplify database operations while enhancing speed and minimizing direct query usage.
- Integrate Angular or React frameworks to enhance user experience while achieving system compatibility and responsive design.
- Design a Restful API system that supports efficient communication between clients and servers while maintaining functional consistency between web and mobile applications.

3.5 Conclusion

MVC design directly tackles both scalability and maintainability problems that arise in web application development. The platform achieves growth readiness and boosts both developer efficiency and user satisfaction through modular and dynamic development.

4. Architecture for Event Sourcing

4.1 Overview and Key Concepts

In distributed systems Event Sourcing (ES) manages state changes by documenting each change as immutable events in a single append-only log. ES differs from traditional systems by documenting all state changes in its append-only log which becomes the single definitive source of truth. This method improves system adaptability and diagnostic capabilities while improving audit functions through the possibility to recreate former states and test different outcomes [4][5][6].

Key concepts include:

- **Event Log:** Event Log functions as an immutable record for all state changes which guarantees both precise data tracking and comprehensive audit trails[5].
- **Replayability:** Applications can recover to prior states by reprocessing recorded events which facilitates both system restoration and debugging processes [4].
- **DDD Integration:** DDD Integration converts domain events such as "Post Created" into functional representations of business processes [2].
- **Decoupling:** Decoupling separates read and write operations which is frequently combined with CQRS to improve efficiency [4][6].

4.2 Relevance to Application Context.

Event Sourcing meets the platform's requirements because of its advantages:

- **Auditability:** The system logs user behaviours over time to verify data integrity and regulatory compliance [4][5].
- **Resilience:** Event replay methods reduce system downtime and allow malfunction recovery and state restoration [6].
- **The system grows well by separating read and write operations, reducing database load [5].**
- **Debugging:** Detailed event logs enable retroactive analysis and scenario testing [5].

4.3 Performance Analysis

Event Sourcing offers key performance improvements:

Redesigning Web Application Architecture for Scalability and Maintainability: A Case Study

- **Scalability:** The approach separates event storage from projections which allows distributed processing to manage increasing demand effectively [6].
- **Testability:** Testability allows for comprehensive testing through simulated event sequences and scenario analysis[4][5].
- **Data Consistency:** The system maintains strong consistency across event streams but permits projections to eventually reach synchronization [4][6].
- **Collaboration:** Teams can develop different services simultaneously without conflict because parallel development support enables separate work streams [5][6].

4.4 Implementation of the New Version.

To include Event Sourcing into the web application, the following actions have to be followed:

- Maintaining distributed and robust event log systems [4], use Amazon Kinesis and Apache Kafka.
- Create unique read models that increase query performance while preserving data synchronizing [5][6].
- CQRS: Combine CQRS with ES to properly control read/write activities for scalable system performance [5][6].
- Schema Evolution: Allow changes in event structures without disturbance by use of versioned events [4][5].

4.5 Conclusion

Event Sourcing maintains scalable systems while delivering audit trails and reliable operations through its comprehensive recording of all state modifications. The combination of its flexible framework with strong debugging tools renders it both durable and future resistant. ES integrated with CQRS delivers an architecture that maintains scalability and maintainability to support ongoing platform changes [4][5][6].

5. Microservices: Third Chosen Architecture

5.1 Why Microservices Was Chosen

The microservices architecture provides solutions for slow website performance alongside issues of scalability limitations and debugging problems while also reducing merge conflicts. Studies showcase the modular distributed nature of microservices which allows for fault isolation and independent scalability while enhancing team collaboration. These characteristics establish microservices architecture as the best option for evolving the web application.

5.2 Relevance to the Web Application

- **Decoupled Services:** Microservices architecture transforms monolithic systems into smaller, independent business-focused services. Independent development and management of features such as user management and notification systems allow for reduced complexity [7][8].
- **Scalability:** Every service can expand independently which maintains system performance in peak times by allowing separate scaling of components such as notifications [9].
- **Independent Deployments:** Teams can deploy each service independently which prevents system-wide disruptions, thereby minimizing interruptions and resolving merge conflicts more effectively [7][8].
- **Fault Isolation and Debugging:** The modular design enables faster issue identification and defect resolution which significantly decreases system downtime [8][9].

5.3 Key Analysis.

- **Scalability:** Every microservice component scales independently, hence optimizing resources and general performance [7][9].
- **Testability:** By streamlining both testing and debugging procedures producing larger test coverage along with faster problem resolution, modular architecture improves testability [7].

Redesigning Web Application Architecture for Scalability and Maintainability: A Case Study

- Parallel development encourages teams to operate separately, hence reducing disputes and enabling onboarding of new engineers [8].

5.4 Conclusion.

The Microservices design increases maintainability and team cooperation at the same time as it delivers better scalabilities. The modular approach produces a flexible foundation fit for further expansion.

6. Combining MVC, event sourcing, and microservices.

While improving cooperation, MVC combined with Event Sourcing and Microservices offers a strong solution to the scalability and performance issues of the platform.

6.1 Role of Each Architecture

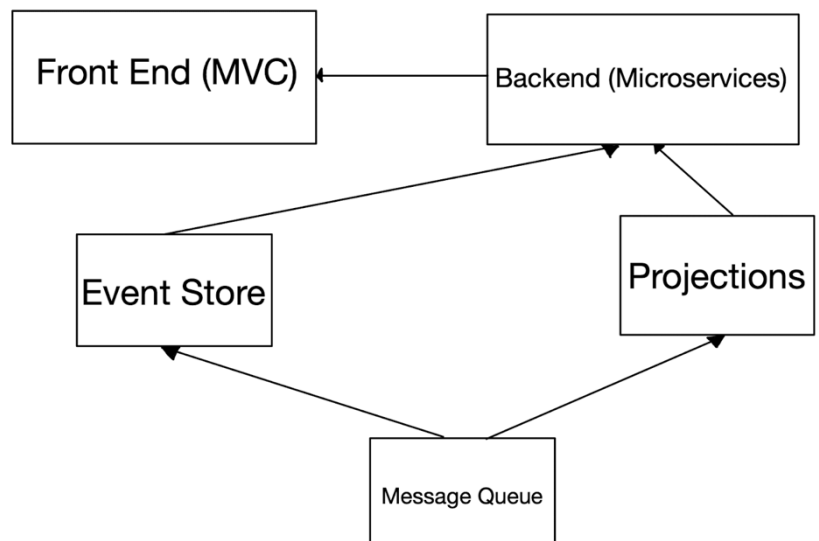
- **MVC:** MVC architecture works by dividing responsibilities within the presentation layer. By separating concerns into distinct roles the Controller processes inputs while the Model generates projections and the View displays data resulting in improved modularity and testability [1][2].
- **Event Sourcing:** Event Sourcing records state transitions as unalterable events which provide audit trails and facilitate system recovery. The generation of "Post Created" events supplies data to both MVC and Microservices architectures which improves system resilience [4][6].
- **Microservices:** The Microservices architecture breaks systems into scalable components like User and Notification Services while processing events through parallel development [7][8][9].

6.2 Understanding Important Features

- **Scalability:** While Microservices provide scalable operations via distinct scaling procedures, Event Sourced activity separation helps to alleviate database load [6][9].
- Microservices allow particular service testing while Event Sourcing offers event replay systems and the MVC framework simplifies UI testing procedures [1][4][8].
- **Collaboration:** Teams develop individual services while Event Sourcing ensures all data remains consistent [8][9].

6.3 Workflow

MVC processes user inputs and triggers events.
Event Sourcing logs these events immutably.
Microservices consume events and execute specific tasks.
Projections supply data for optimized reads.
[1][4][7]



7. This research report concludes.

Web-based systems benefit from scalable solutions through the integration of MVC architecture with Event Sourcing and Microservices. These architectural methods when combined improve scalability and robustness while strengthening modularity which leads to better developer efficiency and user experience.

References

- [1] D.-P. Pop and A. Altar, "Designing an MVC model for rapid web application development," *Procedia Engineering*, vol. 69, pp. 1172–1179, Jan. 2014, doi: 10.1016/j.proeng.2014.03.106.
- [2] A. Majeed, I. Rauf, Korea Aerospace University, and Department of Computer Science, Virtual University, Pakistan, "MVC Architecture: A detailed insight to the modern web applications development," journal-article, 2018. [Online]. Available: <https://crimsonpublishers.com/prsp/pdf/PRSP.000505.pdf>
- [3] S. Necula, "Exploring The Model-View-Controller (MVC) Architecture: A Broad Analysis of Market and Technological Applications," *Exploring the Model-View-Controller (MVC) Architecture: A Broad Analysis of Market and Technological Applications*, Apr. 2024, doi: 10.20944/preprints202404.1860.v1.
- [4] M. Overeem ☆, M. Spoor, S. Jansen, S. Brinkkemper, and The Author(s), "An empirical characterization of event sourced systems and their schema evolution — Lessons from industry." [Online]. Available: <https://doi.org/10.1016/j.jss.2021.110970>
- [5] S. Lima, J. Correia, F. Araujo, and J. Cardoso, "Improving observability in event sourcing systems," *Journal of Systems and Software*, Jun. 2021, doi: 10.1016/j.jss.2021.111015.
- [6] F. Alongi *et al.*, "Event-sourced, observable software architectures: An experience report," journal-article, May 2022. doi: 10.1002/spe.3116.
- [7] J. Esparza-Pedro J., F. D. Muñoz-Escóí, J. M. Bernabéu-Aubán, and Instituto Universitario MixtoTecnológico de Informática; Universitat Politècnica de València, "Modeling microservice architectures," 2024. [Online]. Available: <https://doi.org/10.1016/j.jss.2024.112041>
- [8] P. Di Francesco, I. Malavolta, and P. Lago, "Research on Architecting Microservices: Trends, Focus, and Potential for Industrial Adoption," *Research on Architecting Microservices*, Apr. 2017, doi: 10.1109/icsa.2017.24.
- [9] N. Torvekar and P. S. Game, "Microservices and its Applications An overview," *International Journal of Computer Sciences and Engineering*, vol. 7, no. 4, pp. 803–809, Apr. 2019, doi: 10.26438/ijcse/v7i4.803809.